

Top 10 web hacking techniques of 2022



James Kettle

Director of Research

[@albinowax](#)



Published: Wednesday, 8 February 2023 at 14:20 UTC

Updated: Thursday, 16 February 2023 at 08:24 UTC



Welcome to the Top 10 Web Hacking Techniques of 2022, the [16th edition](#) of our annual community-powered effort to identify the most important and innovative web security research published in the last year.

Since publishing our [call for nominations](#) in January, you've submitted a record 46 nominations, and cast votes to single out 15 final-round candidates. Over the last two weeks, an expert panel of researchers [Nicolas Grégoire](#), [Soroush Dalili](#), [Filedescriptor](#), and [myself](#) have analysed, conferred and voted on the 15 finalists, to bring you the final top 10 new web hacking techniques of 2022. As usual, we haven't excluded our own research, but panellists can't vote for anything they're affiliated with.

This year, for the third year running, there's been a noticeable improvement in the number of quality nominations. While outright novel techniques and class-breaks have gotten rarer, there's more people pushing at the boundaries and sharing their findings than ever. It's great to see the research ecosystem flourishing, even if it makes it harder to select a top ten!

Before we begin the countdown, I should note that any attempt to compress a year of research into a top ten list is going to leave valuable techniques overlooked. If you're hungry for knowledge, I highly recommend reading the [entire nomination list](#).

From the final ten, two key themes stand out - single-sign on, and request smuggling. Let's take a closer look!

10 - Exploiting Web3's Hidden Attack Surface: Universal XSS on Netlify's Next.js Library

As we find ways to shovel ever more complexity into our software, even websites that look static can hide serious vulnerabilities. In [Exploiting Web3's Hidden Attack Surface](#), [Sam Curry](#) and [Shubham Shah](#) tear apart numerous cryptocurrency sites with a blend of [XSS](#), [SSRF](#) and cache poisoning originating from Netlify's Next.js library. We're eager to see if the methodology and cryptocurrency ecosystem insights fuel further discoveries in this field.

9 - Practical client-side path-traversal attacks

Sometimes a vulnerability class can be quite visible, but remain overlooked for years due to low apparent severity.

In [Practical client-side path-traversal attacks](#), [Medi](#) explores a website behaviour that's very common - placing user input inside a request path - and demonstrates a clear pathway to real impact. This behaviour has surfaced in exploit chains a few times over the years but this post shows it's time to recognise it as a vulnerability in its own right. This cousin of client-side parameter pollution has [already inspired a follow-up](#) that uses it for [CSRF](#), and we're sure more will come.

8 - Psychic Signatures in Java

The catchily-named [Psychic Signatures in Java](#) by [Neil Madden](#) shows a critical and really very simple attack using the number 0 to forge ECDSA signatures, undermining the cryptographic foundation of numerous core web technologies including JWT and SAML.

This use of an ancient crypto attack to topple modern web tech is a great reminder that you don't always need a complex attack to achieve massive impact - and as nice as abstractions are, sometimes it's worth looking further down the stack.

7 - Worldwide Server-side Cache Poisoning on All Akamai Edge Nodes

Back in 2019, one of the nominations for the top 10 was an article [theorising about the exploitation potential of HTTP hop-by-hop headers](#), and calling for further research on the topic. Three years later, [Worldwide Server-side Cache Poisoning on All Akamai Edge Nodes](#) makes use of this concept for massive impact and a whole lot of bug bounties, establishing the technique as essential knowledge for web hackers and server implementers alike.

We also appreciate how [Jacopo Tediosi](#) throws some rare light into the world of pain people using advanced techniques can encounter when trying to get their bug bounty reports triaged.

6 - Making HTTP header injection critical via response queue poisoning

Ever wondered why people sometimes inject a HTTP response header, but then refer to it as 'Response Splitting' even though they never actually split the response? In [Making HTTP header injection critical via response queue poisoning](#), I explore the long-forgotten response-splitting technique with a high-impact, high-payout case study.

As noted by [filedescriptor](#), "It seems like there's an infinite amount of anomalies among proxies that bring you an infinite amount of Request Smuggling techniques"... more on that later.

5 - Bypassing .NET Serialization Binders

In 2020, the .NET SerializationBinder documentation changed the statement "SerializationBinder can also be used for security" to "SerializationBinder can not be used for security".

From this simple teaser, [Bypassing .NET Serialization Binders](#) by [Markus Wulfstange](#) delves into why, ultimately building exploits for DevExpress and Microsoft Exchange as case-studies. This research cites an exceptional amount of documentation and related research, making it a great gateway into .NET serialization innards.

Quality research like this often inspires action, and it turns out Soroush has already built on it and integrated the results into [YSoSerial.Net](#). You'll be left wondering what security-critical documentation has changed since you last read it.

4 - Hacking the Cloud with SAML

Just like [OAuth](#), you can't discuss SSO without discussing SAML. While some SAML vulnerabilities are all too well-known, [Hacking the Cloud with SAML](#) by [Felix Wilhelm](#) shines in showing how terrifyingly expansive SAML's attack-surface is. This culminates with an XML document that uses an integer truncation bug to trigger arbitrary bytecode execution when Java attempts to verify its signature.

This is must-watch research. The presentation recording makes the topic exceptionally accessible and provides valuable context, but if you're already familiar with SAML exploitation you might get by with the [slides](#) or [condensed writeup](#).

3 - Zimbra Email - Stealing Clear-Text Credentials via Memcache injection

Application-layer caching is widely used but only rarely exploited. In [Zimbra Email - Stealing Clear-Text Credentials via Memcache injection](#), [Simon Scannell](#) takes this rare bug class to an unseen level by triggering the memcache equivalent of response queue poisoning, causing protocol-level chaos.

This innovative research is a superb demonstration of what can be achieved with deep knowledge of a target. We'll be keeping watch for both server-side cache injection and alternate-protocol desync attacks in future.

2 - Browser-Powered Desync Attacks: A New Frontier in HTTP Request Smuggling

In [Browser-Powered Desync Attacks](#), I explore new vectors for [HTTP Request Smuggling](#), compromising targets ranging from Amazon to Apache and ultimately taking the attack client-side into victim's browsers.

I found this research seriously technically challenging, but thankfully the Web Security Academy team was able to [provide labs](#) to help readers practise, and we've seen people having success with the technique in the wild since. The panel had numerous nice things to say including "The creativity from desync worm (reminiscence of XSS worm) to client-side desync is off the chart" "New concepts, numerous use-cases, impressive as usual" "also gives the audience/reader things to explore".

Request smuggling has proved itself an unstoppable source of novel threats for several years running now, and with plenty of unexplored avenues this will likely continue until HTTP/1 has been fully stamped out, which I imagine will take a while.

1 - Account hijacking using dirty dancing in sign-in OAuth-flows

OAuth has become the foundation of modern SSO, and attacks on it have been a hacker staple ever since. Recently, browsers introduced referrer-stripping mitigations which thwarted one of the most popular techniques - or so we thought. In [Account hijacking using dirty dancing in sign-in OAuth-flows](#), [Frans Rosen](#) shows that modern web stacks provide ample alternatives.

This must-read research provides a masterclass in chaining OAuth quirks with low-impact URL-leak gadgets including promiscuous postMessages, third-party XSS and URL storage. Many of these bugs

would previously have been dismissed as having no significant security impact, so they've had years to proliferate.

The sheer potential of these attacks quickly inspired us to take a stab at [enabling automated URL-leak detection](#), and we're also exploring integrating these techniques into labs for our [OAuth Academy topic](#) to help the community get practical experience applying them.

This is an outstanding piece of research that we expect to yield fruit for years to come. Congratulations to Frans for a well deserved win!

Conclusion

In 2022 the community published more outstanding research than ever, leading to a broad vote spread, and fierce competition for the top 10 slots. Condensing 40+ nominations into a top 10 list does an inevitable injustice to the 30 runners up so we recommend enthusiasts read the [entire nomination list](#). Also, if your own favourite didn't make it, feel free to tweet us to let us know!

Part of what lands an entry in the top 10 is its expected longevity, so it's well worth getting caught up with [past year's top 10s](#) too. If you're interested in getting a preview of what might win in 2023, you can subscribe to [@PortSwiggerRes](#), [r/websecurityresearch](#), and [@portswiggerres@infosec.exchange](#).

If you're interested in doing this kind of research yourself, I've shared a few lessons I've learned over the years in [Hunting Evasive Vulnerabilities](#), and [So you want to be a web security researcher?](#) Also, if you're already putting out this kind of work, I should also mention that [we're looking to hire another researcher](#).

Thanks again to everyone who took part! Without your nominations, votes, and most-importantly research, this wouldn't be possible.

Till next time!

[top-10-techniques](#)

[Top 10 Hacking Techniques](#)

[← Back to all articles](#)

Related Research



Burp Suite

Web vulnerability scanner
Burp Suite Editions
Release Notes

Vulnerabilities

Cross-site scripting (XSS)
SQL injection
Cross-site request forgery
XML external entity injection
Directory traversal
Server-side request forgery

Customers

Organizations
Testers
Developers

Company

About
Careers
Contact
Legal
Privacy Notice
Modern Slavery Statement

Insights

Web Security Academy
Blog
Research
Engineering



© 2026 PortSwigger Ltd.

